1. System Structure and Technology Responsibilities

The system will maintain structured football knowledge, execute topic-specific analyses and generate team-specific strategic options.

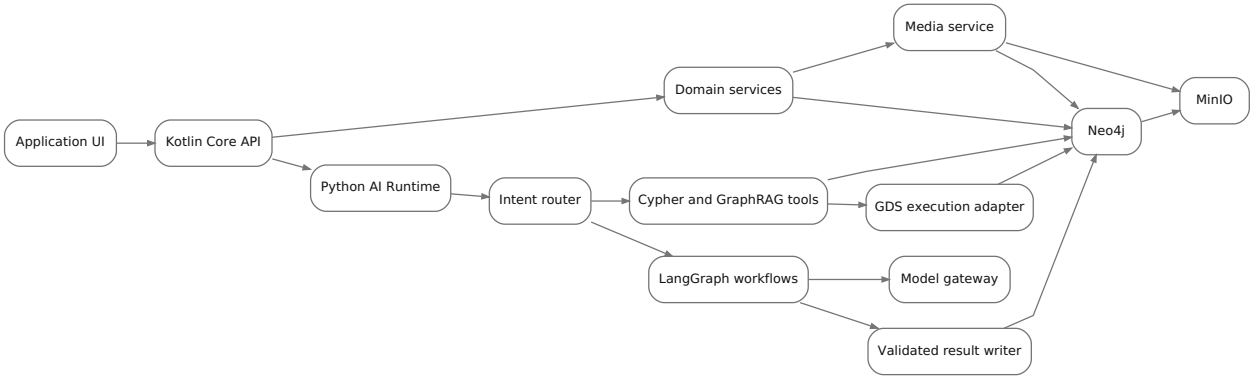
The implementation should begin with two primary application runtimes:

- **Kotlin Core:** Application APIs, authentication, domain commands, roster management, playbook management, media access and review operations.
- **Python AI Runtime:** Intent detection, retrieval orchestration, analysis execution, model integration and LangGraph workflows.

Both runtimes will access Neo4j through defined repository and tool interfaces. Football records must have a designated writer to prevent conflicting implementations of the same business rules. Media objects in MinIO must always be represented by graph-linked asset metadata in Neo4j so the platform can trace which media generated which analyses, what evidence was used and which downstream outputs were derived from that source.

Technology	Responsibility	Usage context
Neo4j	Authoritative football data and relationships	Players, playbooks, observations, analyses and strategies
Cypher	Structured retrieval, aggregation and graph traversal	Exact queries, statistics, constraints and dependencies
Neo4j Vector Index	Semantic candidate retrieval	Coaching notes, play explanations and expert knowledge
Neo4j GDS	Graph algorithm execution	Similarity, clustering and validated graph analytics
MinIO	Binary object storage	Videos, diagrams, images and generated artifacts
LLM	Interpretation, classification and synthesis	Football analysis, explanation and strategy generation
LangGraph	Stateful workflow orchestration	Multi-step analysis, validation and coach revision
Kotlin	Core application services	Typed APIs, domain logic and user-facing operations
Python	AI and analytical implementation	Model adapters, retrieval tools and workflow nodes

Proposed runtime architecture:



The diagram represents logical responsibilities. Each box does not require its own deployment.

---

2. Neo4j Data Modeling Rules

Neo4j will store both the operational football knowledge and the structured outputs produced from it.

The initial model should use separate entity families for:

- Identity and membership.
- Physical attributes and technical skills.
- Playbooks and assignments.
- Opponent observations.
- Analysis definitions and execution results.
- Metrics and validation.
- Strategy candidates and coaching decisions.
- Media references, analysis lineage and provenance.

Common properties

Every persistent domain record should carry:

- `id`: Stable application identifier.
- `tenant_id`: Owning team workspace.
- `schema_version`: Data-contract version.
- `created_at`: Creation timestamp.
- `created_by`: User or system identity.
- `status`: Domain-specific lifecycle status.

Versioned records should additionally include:

- `revision_id`
- `revision_number`
- `effective_from`
- `effective_to`, where applicable

- `recorded_at`
- `supersedes_revision_id`

Business history and recording history must remain distinguishable. A roster correction entered today may describe a change that became effective earlier.

Graph constraints

Implement uniqueness constraints for application identifiers and appropriate composite identifiers. Validate relationship endpoints and tenant ownership in the application layer.

Avoid putting every property into its own node. Use nodes when a concept needs independent identity, evidence, history or relationships.

---

3. Player, Membership and Availability Model

A player’s identity must be separated from their membership and current availability.

Player

Stores stable identity and profile information.

TeamMembership

Connects a player to a team and season, with joining and departure dates.

PositionAssignment

Represents a primary position, secondary position or situational role.

AvailabilityRevision

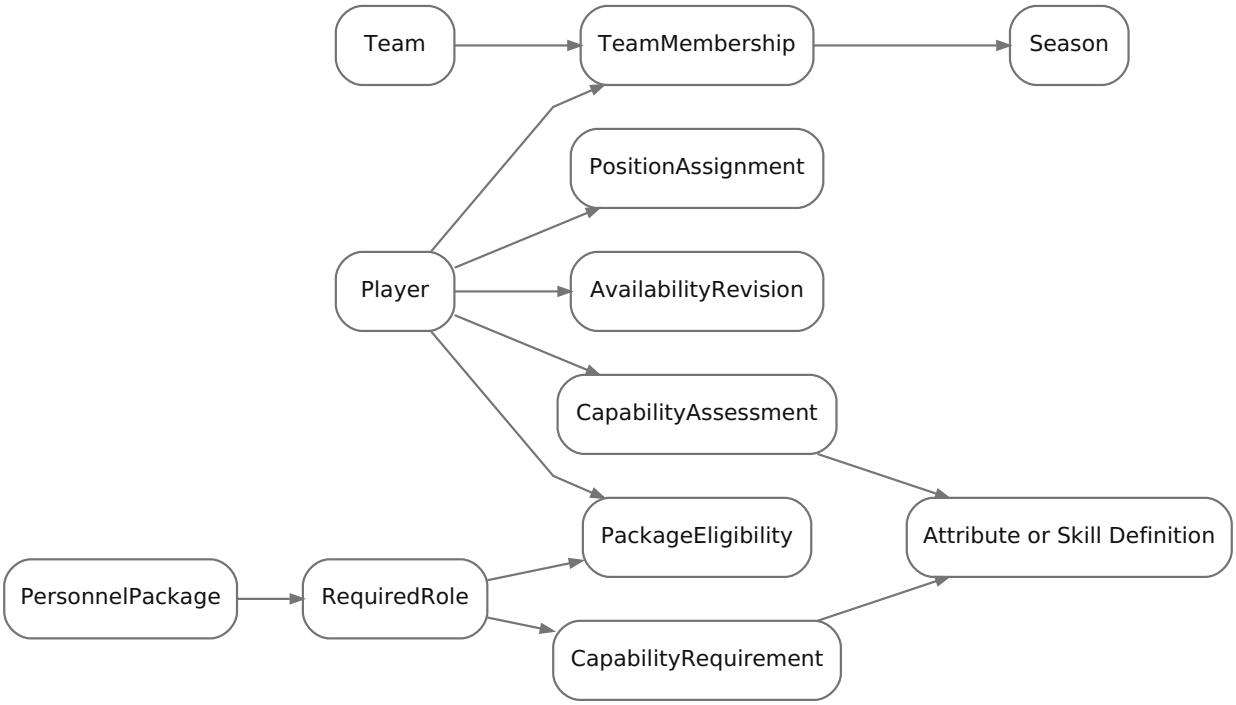
Records availability and participation restrictions supplied by authorized staff.

PersonnelPackage

Defines a collection of required roles.

PackageEligibility

Connects a player to a package role under explicit conditions.



A player becoming unavailable should not erase their capability assessments. It changes eligibility for applicable dates and scenarios.

Historical queries must return the roster and availability that applied at the requested time.

---

4. Attribute, Skill and Assessment Definitions

The platform requires a configurable capability catalog.

AttributeDefinition

Defines a physical or measurable property:

- Name and description.
- Measurement unit.
- Accepted value range.
- Measurement protocol.
- Applicable positions.
- Interpretation rules.

SkillDefinition

Defines an execution capability:

- Technique or task.
- Observable behaviors.
- Assessment rubric.
- Applicable football context.
- Required evidence.

- Scale and interpretation.

CapabilityAssessment

Stores an evaluation against a definition:

- Player and definition reference.
- Value or score.
- Assessment context.
- Measurement or evaluation method.
- Source and evaluator.
- Effective date.
- Evidence references.
- Confidence, if the method supports it.

Measured speed, coach-rated speed and estimated speed from film must remain separate assessments.

The system should preserve `unknown`, `not_assessed` and `not_applicable` states. Missing information must not become a zero score.

---

5. Metric and Score Architecture

Scores will use explicit definitions rather than a generic `score` field shared across unrelated tasks.

Metric family	Example	Interpretation
Physical measurement	Sprint time	Measured value with a unit
Skill assessment	Pass-protection technique rating	Rubric-based execution evaluation
Tactical assessment	Read-progression competency	Context-specific assignment evaluation
Classification confidence	Confidence in a coverage label	Estimated uncertainty of a prediction
Validation score	Expert review result	Performance against a defined rubric
Evaluation accuracy	Correct classifications over a test set	Dataset-level model performance
Strategy suitability	Personnel compatibility	Suitability under selected conditions

MetricDefinition fields

- `metric_id`
- `name`
- `metric_type`
- `unit`
- `scale_min` and `scale_max`, where relevant
- `higher_is_better`
- `rubric_version`

- `calculation_method`
- `applicability_rules`

MetricValue fields

- Target entity or analysis result.
- Metric definition.
- Value.
- Sample size, where relevant.
- Evaluation context.
- Method version.
- Evidence references.

Accuracy must be calculated against reference labels. A model’s self-reported confidence must not be stored as measured accuracy.

---

6. Playbook and Assignment Schema

A play must be represented as structured football logic connected to its visual diagram.

Play

Stable identity of a team play.

PlayVersion

A published implementation of that play.

Formation

Initial structure, alignment and eligible roles.

Assignment

A responsibility attached to a role within a play version.

ReadRule

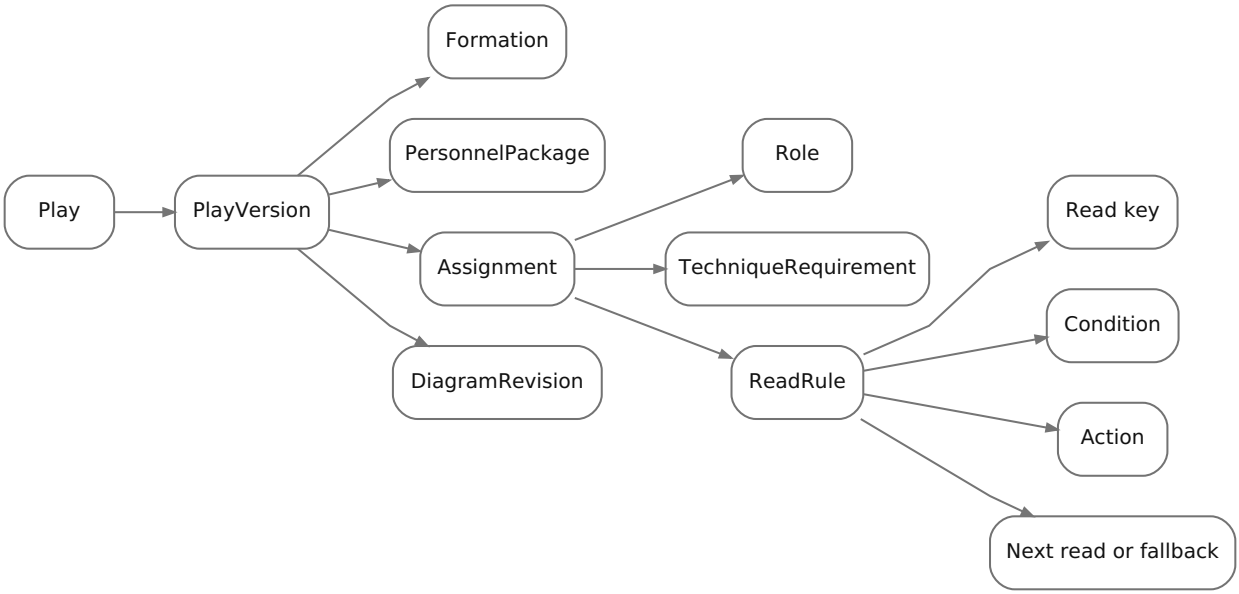
A conditional decision that determines an action.

TechniqueRequirement

A technique needed to execute an assignment.

Adjustment

A modification activated by an explicit condition or coaching decision.



Each assignment should define:

- Starting alignment.
- Initial action.
- Required technique.
- Timing or sequence.
- Read target.
- Conditional response.
- Dependencies on other assignments.
- Exception or fallback.

A prose explanation remains useful, but the runtime should not need to reinterpret the entire explanation every time it evaluates the play.

---

7. Diagram Editor and Visual Data Contract

The diagram editor will produce a structured representation alongside the rendered image.

Editable diagram data

- Field coordinate system.
- Offensive and defensive player markers.
- Role identifiers.
- Alignment coordinates.
- Routes and movement paths.
- Blocking paths and interaction targets.
- Motion segments.
- Timing or phase information.
- Assignment and read annotations.

Storage

- Diagram metadata and football relationships: Neo4j.
- Rendered image or export: MinIO.
- Large geometry payloads: MinIO when appropriate, with an immutable reference from Neo4j.
- Query-relevant geometry and role links: Neo4j.
- Media-to-analysis lineage: Neo4j relationships linking each diagram or video asset to the analyses, evidence links and derived results created from it.

Coach-created and automatically reconstructed diagrams should share the same import/export contract.

Observed trajectories and intended assignments must carry different semantic types. A video-derived path is not automatically proof of the intended assignment.

---

8. MinIO Object Lifecycle

MinIO will store binary objects. Neo4j will maintain the application’s object references and football context.

MediaAsset fields

- `asset_id`
- `tenant_id`
- `bucket`
- `object_key`
- `object_version`, when enabled
- `content_type`
- `size_bytes`
- `content_hash`
- `processing_status`
- `source_asset_id`, for derived objects
- `origin_type`, such as uploaded, imported, generated or derived
- `source_system`, when the asset originated outside the platform
- `captured_at`, when known
- `analysis_ready`, indicating whether the asset is eligible as an analysis input

Upload sequence

1. The client requests an upload session.
2. The API checks permissions and creates a pending asset record.
3. The media service provides an authorized upload mechanism.
4. The client uploads the object.
5. Finalization verifies existence, size, type and integrity.
6. The asset becomes available for ingestion or analysis.

Neo4j linkage requirements

Each persisted object in MinIO must have a corresponding graph record and relationships that preserve football context and lineage. At minimum, the model should support:

- `(MediaAsset)-[:ABOUT]->(Play | PlayVersion | Observation | DiagramRevision | OpponentGame)`
- `(MediaAsset)-[:DERIVED_FROM]->(MediaAsset)` for transformed clips, thumbnails, exports and reconstructed diagrams
- `(AnalysisExecution)-[:USED_SOURCE]->(MediaAsset)` to record which media files were analyzed
- `(AnalysisExecution)-[:PRODUCED]->(AnalysisResult)` to preserve execution lineage
- `(AnalysisResult)-[:SUPPORTED_BY]->(MediaAsset)` or an equivalent evidence-link structure

This linkage must allow a coach or system operator to start from a media asset and answer: which analyses were run from it, which result versions were produced, and what source evidence supports each conclusion.

MinIO's Java SDK supports object operations, presigned URLs and bucket-versioning operations, making it suitable for a Kotlin/JVM media-service adapter. MinIO AIStor Documentation

Do not store expiring presigned URLs as permanent graph references. Generate them when an authorized user requests access.

Failed uploads and orphaned objects require a reconciliation process.

---

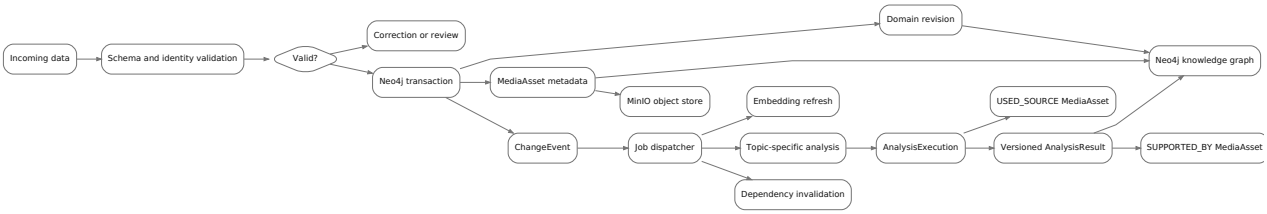
9. Ingestion Pipeline and Change Events

Input sources will include:

- Structured application forms.
- Playbook diagrams and explanations.
- Analyst observations.
- CSV or JSON imports.
- Video-analysis outputs.
- Expert corrections.
- Coaching updates.

Processing stages

1. Validate the input schema.
2. Resolve existing entity identities.
3. Normalize terminology and units.
4. Validate references and scope.
5. Create the appropriate domain revision.
6. Commit the record and a change event.
7. Trigger dependent processing, including any media-derived analysis jobs and lineage updates.



A proposed transactional-outbox pattern can store the domain update and `ChangeEvent` in the same Neo4j transaction. A dispatcher delivers the event to workers.

Consumers must be idempotent because delivery may occur more than once.

---

10. Analysis Topic Registry

Every analysis will be defined as an executable topic.

An `AnalysisDefinition` should contain:

- `analysis_type`
- Football domain.
- Supported target types.
- Required inputs.
- Optional inputs.
- Supported source types, including whether media assets are valid direct inputs.
- Dependency definitions.
- Execution method.
- Output schema.
- Metric definitions.
- Validation rules.
- Refresh triggers.
- Definition version.

Example topic families

Topic	Inputs	Primary output
Coverage classification	Defender movement and context	Candidate coverage classes
Technique assessment	Player actions and rubric	Technique metrics
Personnel fit	Capability assessments and role requirements	Compatibility evaluation
Opponent tendency	Filtered play observations	Frequencies and sample sizes
Protection analysis	Blocking behavior and pressure	Protection interpretation
Strategy validation	Candidate and team constraints	Violations and suitability metrics

One input may trigger multiple topics. Each topic must produce its own result rather than placing unrelated findings in a single unstructured response.

11. Analysis Result Schema

The common result envelope should include:

- `analysis_result_id`
- `analysis_execution_id`
- `analysis_type`
- `definition_version`
- `target_refs`
- `input_snapshot_id`
- `source_media_refs`
- `source_revision_refs`
- `execution_method`
- `category_labels`
- `class_predictions`
- `metric_values`
- `supporting_evidence`
- `contradicting_evidence`
- `evidence_asset_refs`
- `missing_inputs`
- `validation_status`
- `created_at`

Topic-specific fields will extend this envelope.

For example, a coverage analysis may produce several class hypotheses with confidence values. A player assessment may produce a technique score and rubric findings. A tendency analysis may produce counts, percentages and denominators.

Keep these states independent:

- **Execution:** Did the job complete?
- **Validation:** Has the finding passed review?
- **Freshness:** Are the referenced inputs still applicable?

A completed analysis may still be unreviewed or stale.

Analysis provenance must remain queryable. From any `AnalysisResult`, the platform should be able to resolve the originating execution, the media assets and revisions used as inputs, and any derived media or strategy artifacts created from that result.

---

12. Cypher Query and Constraint Layer

Cypher will execute exact queries, aggregations and explicit graph traversals.

Query tool families

- Player availability.
- Historical roster reconstruction.
- Play and assignment retrieval.
- Role-requirement lookup.
- Opponent situation filtering.
- Tendency aggregation.
- Evidence traversal.
- Media-to-analysis lineage lookup.
- Dependency and invalidation queries.

Each tool should define:

- Typed input parameters.
- Allowed query template.
- Required tenant and date filters.
- Maximum result size.
- Timeout.
- Typed output schema.

LLM-generated parameters must pass validation before execution.

Constraint representation

Store football constraints as typed conditions, such as:

- `requires_available_player`
- `requires_skill_level`
- `prohibits_technique`
- `requires_installed_play_version`
- `applies_to_game_plan`

Interpret these through a controlled rule evaluator. Do not execute arbitrary code stored in coaching notes or graph properties.

---

13. Neo4j Vector Index and Embedding Lifecycle

Vector search will retrieve semantically relevant text within Neo4j.

Embedding targets

- Play explanations.
- Assignment descriptions.
- Coaching notes.
- Expert knowledge passages.
- Reviewed analysis summaries.

Each searchable unit should reference its exact source revision. If an embedding was generated from a media-derived transcript, caption or analysis summary, the corresponding MediaAsset and AnalysisResult lineage should remain linked in Neo4j.

Embedding metadata

- `embedding_model`
- `embedding_dimension`
- `embedding_version`
- `source_revision_id`
- `content_hash`
- `created_at`

Generate a new embedding when the searchable content changes. Structured changes that do not affect the text need not trigger unnecessary embedding work.

Do not mix incompatible embedding models in the same retrieval space.

Neo4j vector indexes support approximate nearest-neighbor retrieval. Query syntax and filtering capabilities depend on the selected Neo4j version, so the implementation must pin and test its supported version. Cypher Manual

Vector similarity measures semantic proximity. It does not establish football correctness or strategic suitability.

---

14. Neo4j GDS Execution Lifecycle

GDS will run graph algorithms against a task-specific projected graph.

Execution sequence

1. Select the analysis definition.
2. Resolve the relevant data revisions.
3. Build a projection containing the required nodes and relationships.
4. Prepare features and missing-value handling.
5. Estimate resource requirements.
6. Execute the selected algorithm.
7. Convert results into the domain analysis schema.
8. Persist versioned results.
9. Release the projection when no longer required.

Candidate applications

- Capability-profile similarity.
- Situation-pattern clustering.
- Concept relationship analysis.
- Opponent behavior grouping.

GDS distinguishes `stream`, `stats`, `mutate` and `write` modes. In particular, `mutate` updates the in-memory graph, while supported `write` operations persist results to Neo4j. Neo4j Graph Data Science

For this platform, controlled result persistence is preferable when provenance and versioning are required. Avoid overwriting authoritative player assessments with raw algorithm output.

---

15. GraphRAG Retrieval Pipeline

GraphRAG will assemble the context needed for a specific task.

Retrieval process

1. Resolve named entities and team terminology.
2. Identify the requested scope and date.
3. Run exact Cypher queries where possible.
4. Use semantic retrieval for relevant explanations and notes.
5. Expand candidate results through approved graph relationships.
6. Apply permissions, revision and relevance filters.
7. Deduplicate and rank evidence.
8. Construct a bounded evidence package.

Evidence package

- Structured facts.
- Relevant assignments and reads.
- Applicable constraints.
- Player assessments.
- Opponent observations.
- Calculated metrics.
- Supporting and contradictory sources.
- Missing information.

Tenant restrictions must apply before unauthorized content reaches the model. Vector post-filtering alone should not serve as the security boundary.

Aggregate questions require queries over the full defined dataset, not counts from the top semantic search results.

---

16. Intent Detection and Execution Planning

Use a two-level classification structure.

Level 1 — Operation

- Retrieve.
- Analyze.

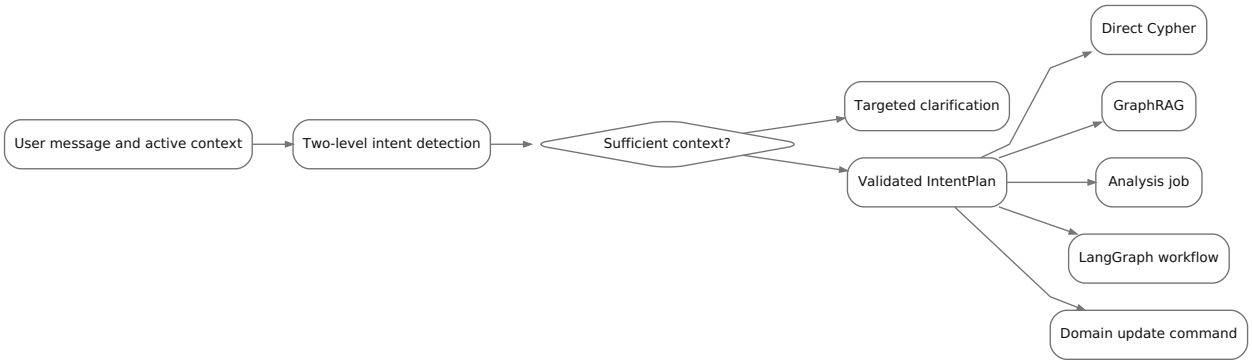
- Compare.
- Generate.
- Modify.
- Update.

Level 2 — Domain and target

- Offense, defense or personnel.
- Player, playbook, opponent, technique or strategy.
- Current, historical or hypothetical context.
- Single play, situation or game-plan scope.

The output should be an `IntentPlan` containing:

- Operation and domain.
- Resolved entity references.
- Temporal scope.
- Execution route.
- Required tools or skills.
- Missing parameters.
- Write intent.
- Classification confidence.



A request may contain several operations. The planner should represent dependencies between them, such as updating availability before regenerating a strategy.

---

17. Agent and Skill Registry

`agent.md` and `skill.md` will be maintained as versioned specifications.

They must be loaded by an explicit runtime implementation.

Agent specification

- Agent identifier.
- Football domain.
- Responsibility.

- Allowed skills.
- Allowed tools.
- Input and output schemas.
- Execution limits.
- Escalation rules.

Skill specification

- Task definition.
- Preconditions.
- Required retrieval.
- Football rules.
- Output contract.
- Validation checks.
- Failure behavior.
- Test cases.

Registry implementation

- Parse and validate specifications.
- Resolve skill dependencies.
- Pin versions for each execution.
- Render prompts.
- Enforce tool permissions.
- Validate model outputs.
- Record the selected specification versions.

Team-specific preferences should be retrieved as data. They should not require copying and editing a separate agent prompt for every team.

---

18. LLM Gateway and Output Validation

All model calls should pass through a common gateway.

Gateway responsibilities

- Task-based model selection.
- Provider adapter.
- Prompt assembly.
- Context and token limits.
- Timeout and retry handling.
- Structured-output validation.
- Usage tracking.
- Trace correlation.
- Model-version recording.

Output validation stages

1. Parse the response.
2. Validate its schema.
3. Verify referenced entity identifiers.
4. Verify that cited evidence exists.
5. Apply relevant football constraints.
6. Detect missing required fields.
7. Route invalid output to a bounded repair attempt or review.

A syntactically valid response is not automatically a valid football conclusion.

If the model lacks sufficient evidence, the output should describe missing inputs instead of manufacturing precision.

---

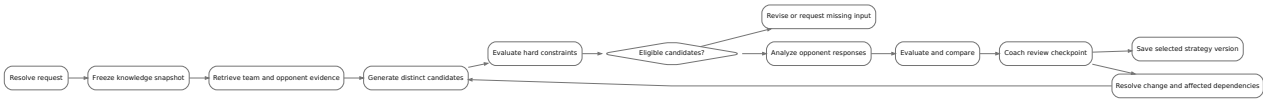
19. LangGraph Strategy Workflow

LangGraph will orchestrate strategic tasks that require state, branching and iteration.

Workflow state

- Request and conversation identifiers.
- Team and opponent references.
- Knowledge snapshot.
- Evidence references.
- Candidate strategy identifiers.
- Constraint results.
- Evaluation metrics.
- Review state.
- Retry and iteration budgets.

Execution flow



LangGraph provides checkpoint and interrupt mechanisms for pausing and resuming workflows. Application logic must still determine whether the external football data has changed and which steps require recomputation. GitHub

Bound iteration counts and terminate with a clear unresolved state when constraints cannot be satisfied.

---

20. Strategy Generation and Ranking

Strategy generation must combine:

- Available personnel.
- Current capability assessments.
- Installed playbook versions.
- Team technique preferences.
- Opponent observations.
- Situation-specific objectives.

Candidates should differ in tactical mechanism, personnel use or adjustment structure.

Hard constraints

Determine whether a candidate is allowed:

- Player availability.
- Required role coverage.
- Prohibited technique.
- Applicable playbook or installation policy.

Soft criteria

Support comparison:

- Personnel fit.
- Evidence strength.
- Execution complexity.
- Installation burden.
- Expected exposure to counters.

Store criterion-level values and explanations. If a composite ranking is used, its weights and normalization must be versioned.

The policy for proposing entirely new plays or techniques remains configurable. Such candidates must explicitly indicate additional installation requirements.

---

21. Coach Changes and Dependency Invalidation

The update workflow must identify both the meaning and scope of a coaching change.

Supported change types

- Terminology alias.
- Assignment correction.
- Technique restriction.
- Player assessment update.
- Availability change.

- Scenario-specific override.
- Game-plan preference.
- Persistent team rule.

Processing

1. Create a typed change proposal.
2. Resolve the target and scope.
3. Validate permissions and conflicts.
4. Commit a new revision.
5. Emit a change event.
6. Locate dependent analyses and strategies.
7. Mark affected results stale.
8. Schedule required recalculation.

Dependencies should be recorded explicitly through input references and relationships.

For example, changing a player's availability invalidates personnel selection. Renaming a play usually changes presentation and retrieval aliases without requiring all tactical analysis to rerun.

---

22. API Contracts and Job Management

The API should distinguish domain commands, queries and asynchronous jobs.

Operation	Proposed endpoint
Create capability assessment	`POST /players/{id}/assessments`
Change availability	`POST /players/{id}/availability-revisions`
Publish play version	`POST /plays/{id}/versions`
Register opponent observation	`POST /games/{id}/observations`
Start analysis	`POST /analysis-jobs`
Submit conversational request	`POST /conversations/{id}/messages`
Generate strategy candidates	`POST /strategy-jobs`
Revise strategy	`POST /strategies/{id}/revisions`
Review result	`POST /analysis-results/{id}/reviews`

Long-running operations

Return `202 Accepted` with a `job_id`.

Expose:

- Job status.
- Progress events.

- Result references.
- Cancellation.
- Failure details.
- Retry eligibility.

Use an idempotency key for repeatable commands and version checks for concurrent edits.

Recommended job states:

`QUEUED`, `RUNNING`, `WAITING_FOR_INPUT`, `WAITING_FOR_REVIEW`, `SUCCEEDED`, `FAILED`, `CANCELLED`.

The queue technology and production checkpoint backend remain implementation decisions. They do not change Neo4j's ownership of football data.

---

23. Development Modules and Dependencies

Module	Implementation scope	Depends on
Domain foundation	IDs, schemas, validation and Neo4j constraints	None
Player domain	Membership, availability and assessments	Domain foundation
Playbook domain	Editor contracts, assignments, reads and versions	Domain foundation
Media service	MinIO upload, retrieval and object references	Identity and permissions
Opponent domain	Games, plays, observations and evidence	Domain foundation, media
Analysis registry	Definitions, result schemas and metrics	Domain model
Cypher tools	Exact queries and deterministic calculations	Graph schema
Embedding service	Content preparation and vector-index lifecycle	Versioned content
GDS adapter	Projections, algorithm execution and result mapping	Analysis registry
GraphRAG	Retrieval, graph expansion and evidence packages	Cypher tools, embeddings
Intent router	Task classification and execution plans	Tool and skill registries
Strategy workflow	Generation, validation and comparison	GraphRAG, capability and playbook models
Coaching adaptation	Scoped updates and invalidation	Revision and dependency model
Evaluation harness	Dataset runs, scoring and regression reports	Analysis contracts
Application UI	Domain editors, chat and review surfaces	Published API contracts

Develop the core against structured data first so that automatic video analysis can integrate through the same contracts.

---

24. Test Coverage and Release Requirements

Domain tests

- Historical roster queries return the correct membership.
- Missing assessments remain unknown.
- Published play revisions preserve prior assignments.
- Scenario overrides do not modify permanent team rules.

Retrieval tests

- Correct team, date and revision filtering.
- Relevant evidence retrieval.
- Accurate aggregation denominators.
- No unauthorized data in model context.

Analysis tests

- Classification against reference labels.
- Metric calculation against known examples.
- Consistent handling of contradictory evidence.
- Validation of GDS outputs against a baseline.

Workflow tests

- Resume after review.
- Retry after transient failure.
- Duplicate event handling.
- Cancellation.
- Recalculation after a relevant update.

Strategy tests

- No hard-constraint violations.
- Traceable evidence.
- Meaningfully distinct candidates.
- Explicit unknowns and installation requirements.

Each release should pin schema, query, skill, prompt and model versions. Replaying the same input context should reproduce the execution configuration, while allowing for documented model-output variability.